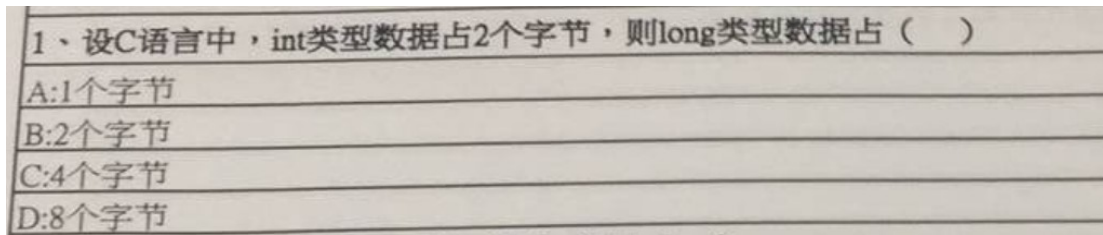


一、选择题

1.



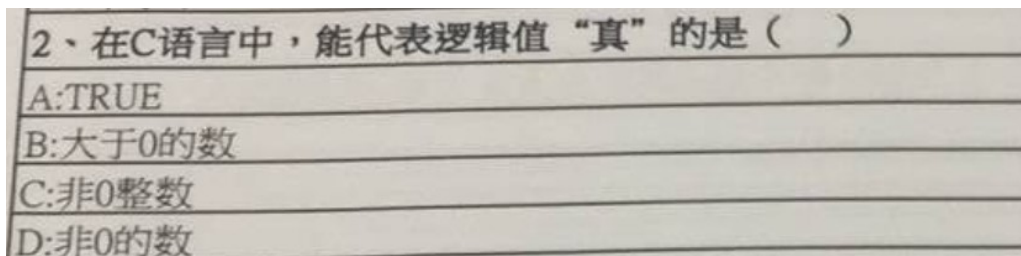
答案：C

分析：int 型为 2 个字节，表明这是 16 位操作系统，在十六位操作系统中 long 型占 4 个字节

知识点：

- 早期的操作系统是 16 位系统：
 - int 用 2 字节表示，范围是 $-32768 \sim 32767$ ；
 - long 用 4 字节表示，范围是 $-2147483648 \sim 2147483647$ 。
- 后来发展到 32 位操作系统，
 - int 用 4 字节表示，与 long 相同。
- 目前的操作系统已发展到 64 位操作系统，但因程序编译工艺的不同，两者表现出不同的差别：
 - 32 位编译系统：int 占四字节，与 long 相同。
 - 64 位编译系统：int 占四字节，long 占 8 字节

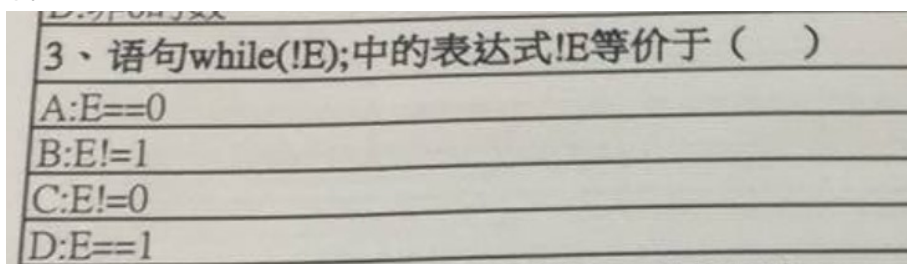
2.



答案：C

分析：在 C 语言中 0 代表假，非 0 的整数代表真，而 TRUE 是面向对象型语言中代表真的关键字，在 C 语言中无意义，而浮点型的数字不能作为逻辑判断量

3.



答案：A

分析：首先 while 的判断为真，即!E 的逻辑结果是真，那么条件!E 等价于 $E==0$ 。! 是取反，如果 E 是 0， $E==0$ 当然成立为真，表达式的值为 1；!E 的值也是 1，所以是等价的；如果 E 是 1， $E==0$ 当然成立为假，表达式的值为 0；!E 的值也是 0，所以是等价的。

知识点:

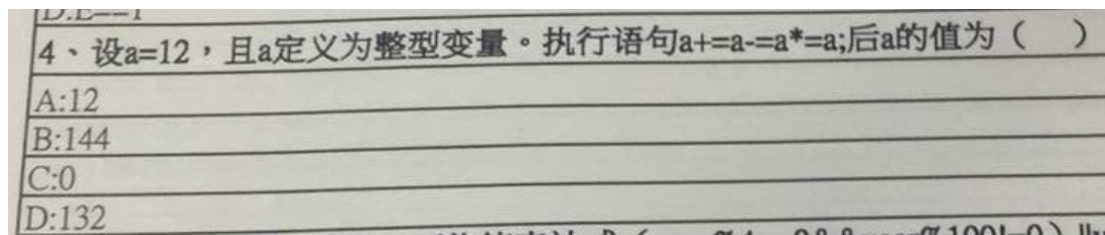
while (!k)

如果k是bool类型,!k等价于k==false (当k等于false时,!k为true,否则!k为false), while(!k)表示直到k等于false时停止循环。

如果k是整型,!k等价于k==0 (当k等于0时,!k为true,否则!k为false), while(!k)表示直到k等于0时停止循环。

如果k是字符型,!k等价于k!='\0' (当k等于'\0'时,!k为true,否则!k为false), while(!k)表示直到k等于'\0'时停止循环。

4.



答案: C

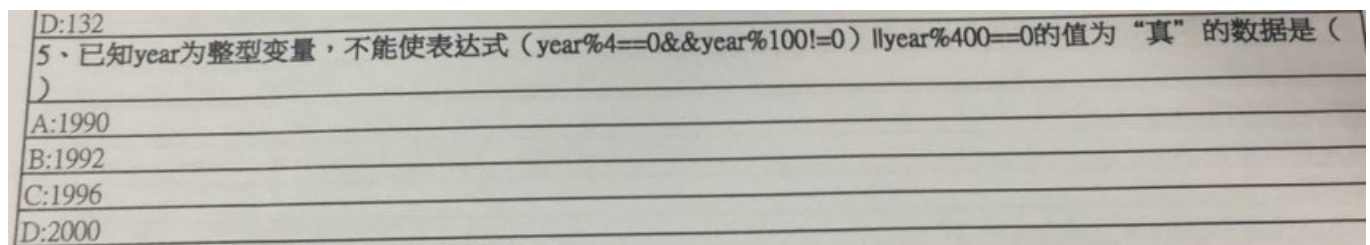
分析: +=, -=, *=优先级是同级的, 但结合性却是自右向左的, 因此执行语句可以拆开成:

1. a = a * a; a = 144;

2. a = a - a; a = 0;

3. a = a + a; a = 0;

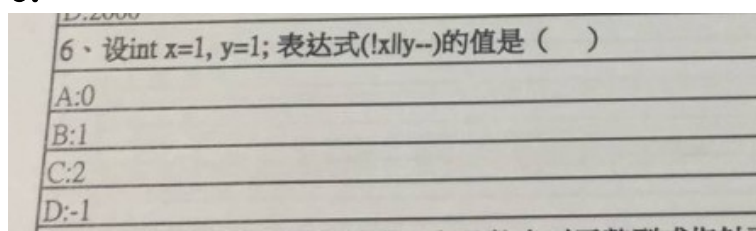
5.



答案: A

分析: 考察优先级, 算术运算优先级高于逻辑运算, 括号的优先级最高。表达式中因为括号的原因,

6.



答案: B

分析: 查考的是运算符的优先级, 因为x为1, 所以!x为0; y减减, 先用再减, 所以值为1, 再进行“或”运算, 即0或1, 故最终值为1, y最后变为0

知识点:

指针最优, 单目运算优于双目运算。

先乘除（模），后加减。

先算术运算，后移位运算，最后位运算。

逻辑运算最后计算。

还有：

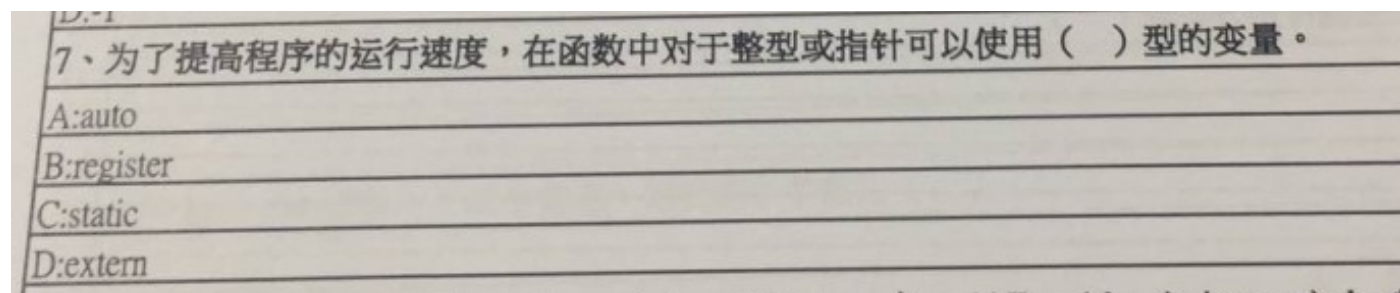
算术运算符最高

关系运算符次之

逻辑运算符最低

例如： $9 > 3 + 4 \&\& 7$ 先算 $3 + 4$ 再判断 9 是不是大于 7，再 $\&\&$

7.



答案：B

分析：考察 C 语言的变量存储类型，register 是寄存器存储变量，是由寄存器分配空间，访问速度比访问内存快，

知识点：

auto	自动变量
register	寄存器变量
extern	外部变量
static	静态变量

自动变量和寄存器变量属于动态存储方式，外部变量和静态变量属于静态存储方式。

auto 局部变量：auto 变量是用堆栈(stack)方式占用存储器空间，因此，当执行此区段是，系统会立即为这个变量分配存储器空间，而程序执行完后，这个堆栈立即被系统收回。在大括号 {} 内声明。

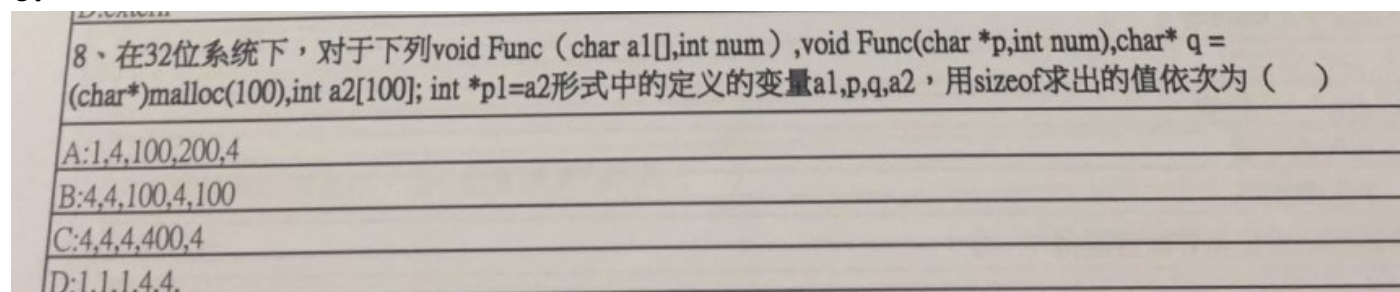
static 静态变量：分配在静态存储区，在程序整个运行期间都不释放

extern 外部申明：为了使变量或者函数除了在定义它的源文件中可以使用外，还可以被其它文件使用

register 寄存器变量：是由寄存器分配空间，访问速度比访问内存快，只能用于整型和字符型变量

register 变量存在 CPU 的寄存器中；auto 类型变量存在内存的栈；static 型的局部变量和全局变量以及 extern 型变量（即全局变量），存在于内存的静态区。

8.

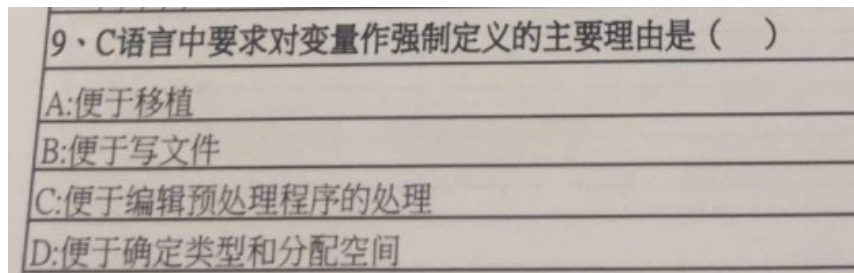


答案：C

分析：32 为系统中，地址的大小为 4 个字节，a1 退化成指针，p、q、p1 为指针，a2 为整型数组，因此答案是 4，4，4，400，4

知识点：指针的大小为系统地址的大小

9.

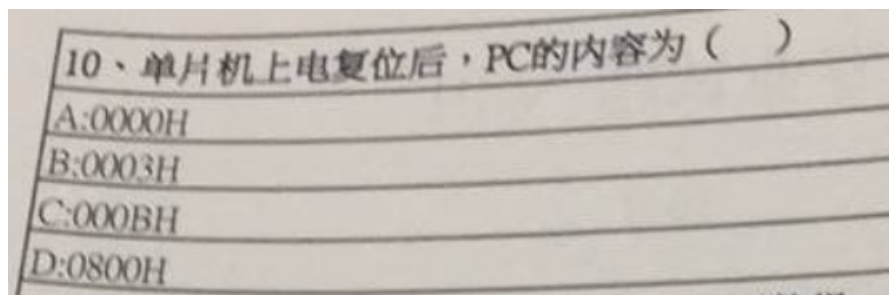


答案：D

分析：A、B：错误。这两个和 C 语言的语言特性完全没有关系。和函数库比较相关。C：错误。事实上，预处理器只通过预处理命令确认标识符（例如宏定义）而完全不检查除此之外的标识符以及它们的类型。剩余的定义和类型检查是预处理阶段结束后翻译阶段生成目标文件代码的同时进行的。

D：正确，只有明确的定义才能保证类型的一致性和分配空间大小的明确性。

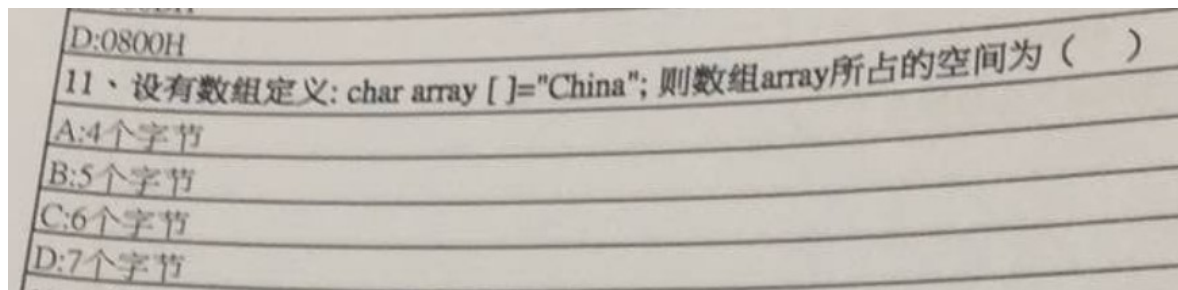
10.



答案：A

分析：复位电路有两种方式：上电自动复位电路和手动复位电路。复位后，程序计数器 PC 变为 0000H，使单片机从程序存储器地址 0000H 单元开始执行。除 P0~P3 为 FFH；SP 为 07H；SBUF 为不确定。大部分特殊功能寄存器清零。

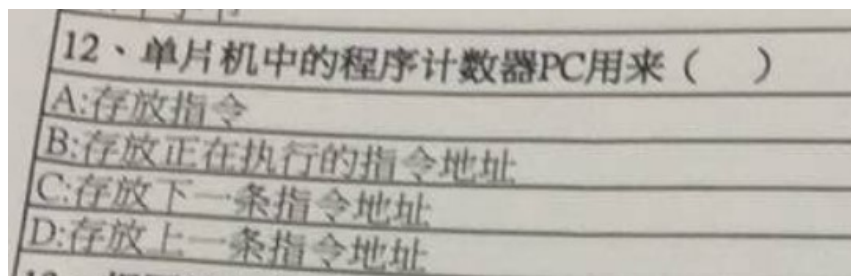
11.



答案：C

分析：字符默认占 1 个字节，china 占 5 个字节，但是空间分配是会在字符数组的后面自动加上 '\0' 字符，所以 array 占 6 个字符

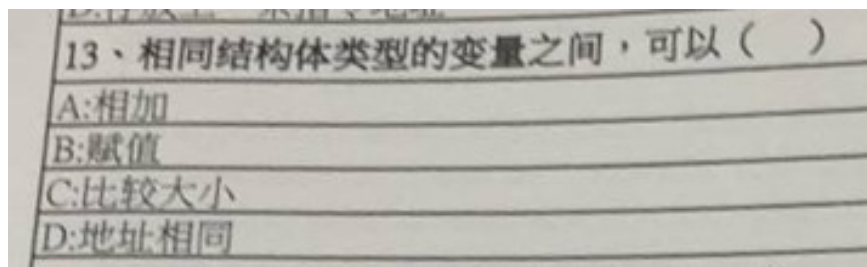
12.



答案: C

分析：程序计算器 PC，也叫程序指针，它存储的当前数值就是将要执行的程序代码的地址，而该程序语句代码被执行后，它的数值马上被加上刚执行的代码所占用的长度，又指向下一个程序代码地址。也有在碰到诸如强制跳转程序代码时，它的数值是赋予一个绝对值。反正它存储的当前数值永远指向将要执行的程序代码的地址。

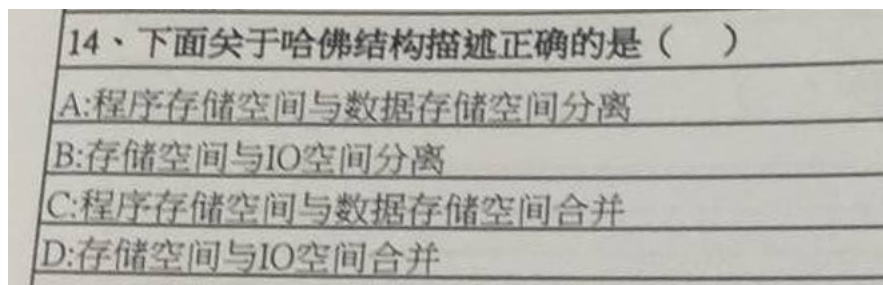
13.



答案: B

分析：结构体类型是一种数据结构，是 C 语言中聚合数据类型结构体可以被声明为变量、指针或数组等，用以实现较复杂的数据结构。结构体同时也是一些元素的集合，这些元素称为结构体的成员，且这些成员可以为不同的类型，成员一般用名字访问。所以结构体无法整体性的相加，比较大小，不同的结构体变量地址也不同

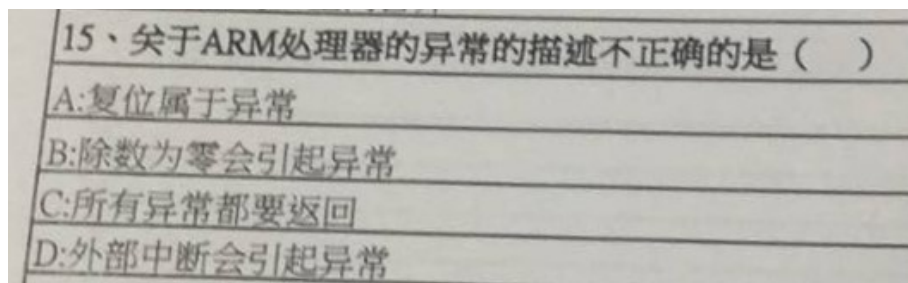
14.



答案: A

分析：哈佛结构是一种将程序指令存储和数据存储分开的存储器结构。哈佛结构是一种并行体系结构，它的主要特点是将程序和数据存储在不同的存储空间中，即程序存储器和数据存储器是两个独立的存储器，每个存储器独立编址、独立访问。哈佛结构与冯·诺依曼结构处理器相比，处理器有两个明显的特点：使用两个独立的存储器模块，分别存储指令和数据，每个存储模块都不允许指令和数据并存；使用独立的两条总线，分别作为 CPU 与每个存储器之间的专用通信路径，而这两条总线之间毫无关联。

15.



答案：B

分析：异常分类

1. 复位异常

当CPU刚上电时或按下 reset 重启键之后进入该异常

2. 一般/快速中断请求

CPU 和外部设备是分别独立的硬件执行单元，一般指外部设备异常

3. 预取指令中止异常

该异常发生在CPU流水线取指阶段，如果目标指令地址是非法地址进入该异常

4. 未定义指令异常

该异常发生在流水线技术里的译码阶段，如果当前指令不能被识别为有效指令，产生未定义指令异常

5. 软件中断指令（swi）异常

该异常是应用程序自己调用时产生的，用于用户程序申请访问硬件资源时

6. 数据中止访问异常

该异常发生在要访问数据地址不存在或者为非法地址时

异常发生的硬件操作：

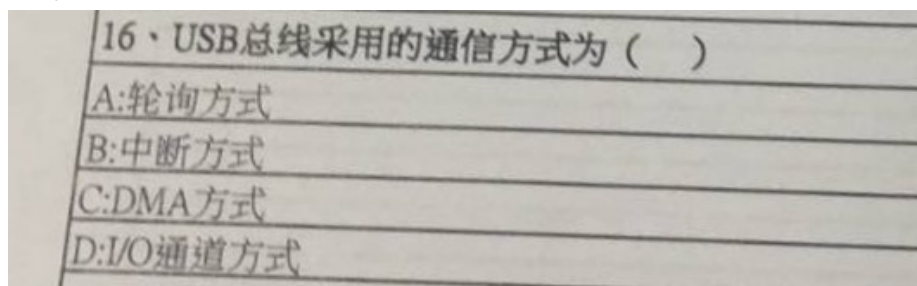
1. 保存执行状态：将CPSR复制到发生的异常模式下SPSR中

2. 模式切换：将CPSR模式位强制设置为与异常类型相对应的值，同时处理器进入到ARM执行模式，禁止所有IRQ中断，当进入FIQ快速中断模式时禁止FIQ中断

3. 保存返回地址：将下一条指令的地址（被打断程序）保存在LR(异常模式下LR_excep)中

4. 跳入异常向量表：强制设置PC的值为相应异常向量地址，跳转到异常处理程序中

16.



答案：B

分析：USB 四种的传输方式：

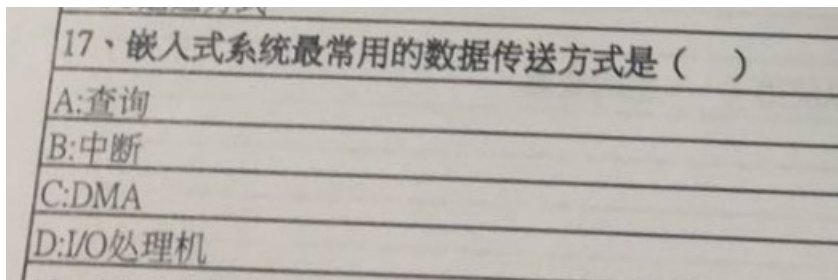
1. 控制(Control)传输

2. 同步(isochronous)传输

3. 中断(interrupt)传输

4. 批量(bulk)传输

17.



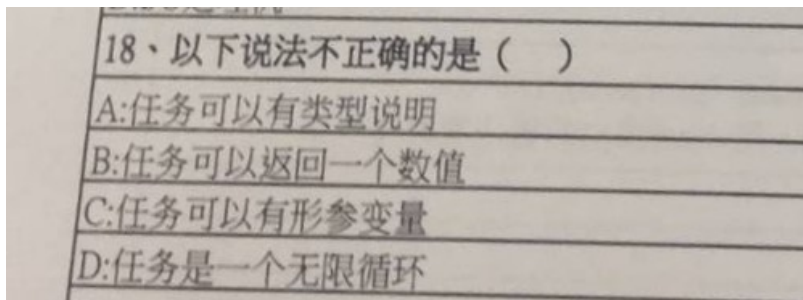
答案：B

分析：设备的连接和工作方式：

直接传送、程序查询、中断、DMA、I/O 处理机

中断方式响应速度快，占用资源少，而嵌入式一般对实时性有要求，而且芯片的资源比较小，需要使用最高效率的代码运行方式，所以一般数据传送使用中断方式，而且芯片本身有中断向量表，调用中断很方便

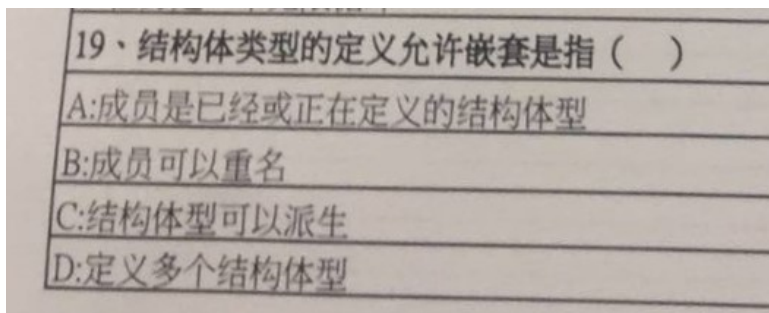
18.



答案：D

分析：程序的整体思路是在主函数 main 中依次放置几个死循环作为任务框架，即每个任务都是一个死循环，利用中断进行任务切换。程序为所有任务设置一个总入口并放在主函数中，串口中断每次返回时必须先经过这个总入口，在总入口处检查任务控制变量（全局变量）的值，任务控制变量已在串口中断中被赋值，其值决定要切换到哪个任务。

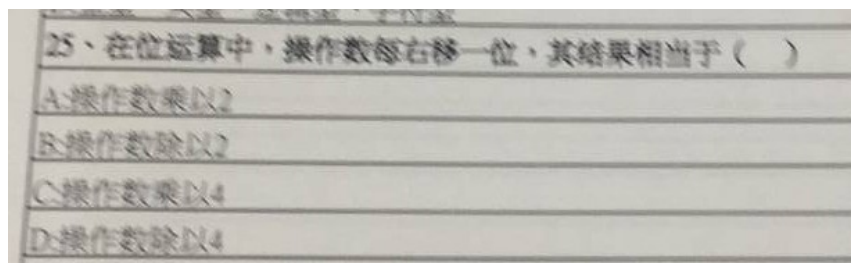
19.



答案：A

分析：结构体中的成员变量在引用时，必须要用结构体变量名+成员变量名来引用，所以成员不可以重名；结构是隐式密封的，不能作为基类在派生出其他的结构，也不能从类派生；定义多个结构体类型与嵌套无关

20.

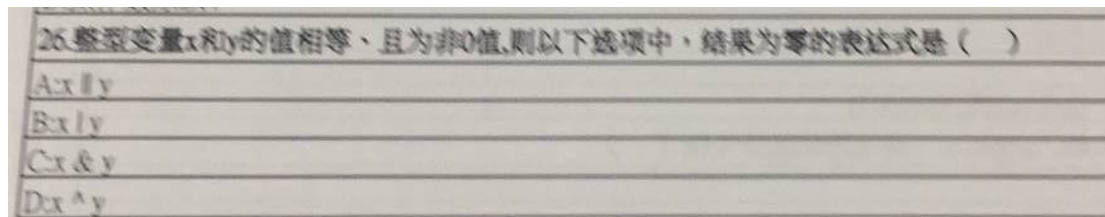


答案: B

分析: 考察位移

举例 0110 -- 6 右移一位 0011 -- 3 相当于操作数除以 2

21.



答案: D

分析: 考察位运算 $x = y \neq 0$

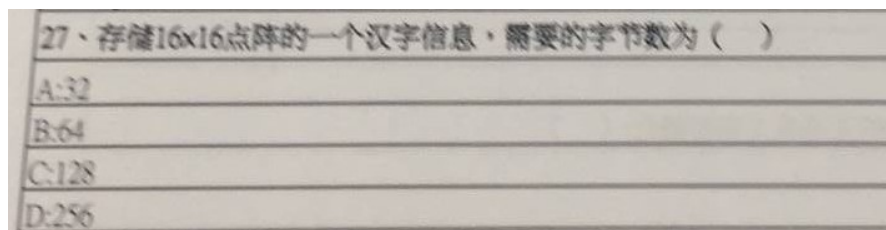
$x || y$ 是逻辑运算只要 x 或 y 不为 0, 则值就不为 0

$x | y$ 是位或运算, 只要操作中有一个不为 0, 则值不为 0

$x \& y$ 是位与运算, 因为 $x = y$, 所以值不为 0

$x \wedge y$ 是位异或运算, 因为 $x = y$, 所以值一定为 0

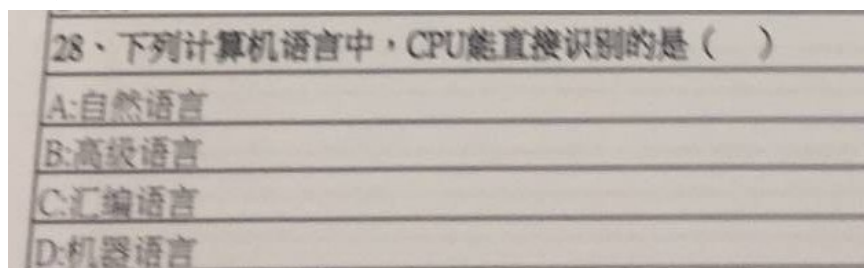
22.



答案: A

分析: 16×16 汉字点阵有 $16 \times 16 = 256$ 位二进制位来表示一个汉字的字形码。在计算机中, 一个字节由 8 位二进制位组成。故一个 16×16 点阵的字形码需以 $256 \text{ 位} / 8 \text{ 位} = 32$ 字节的存储空间存储。

23.



答案: D

分析: CPU 只能执行机器语言程序

24. 以下能正确定义一维数组的选项是()

- A: `int a[5] = {0, 1, 2, 3, 4, 5};`
B: `char a[] = { '0' , '1' , '2' , '3' , '4' , '5' };`
C: `char a = { 'A' , 'B' , 'C' };`
D: `int a[5] = "0123";`

答案: B

分析: A: 数组越界, C: a 不是数组, D: 字符串不能赋值给整型

25. 边沿触发方式的外部中断信号是()有效

- A: 下降沿
B: 上升沿
C: 高电平
D: 低电平

答案: A

分析: 边沿触发方式, 排除 C 和 D。外部终端是负边沿有效所以选 A

知识点:

外部中断有两种触发模式:

- 1、是下降沿触发, 当从高至低电平转变时, 触发产生; 低电平保持多久都只产生一次。所以, 边沿触发适合与以负脉冲形式输入的外部中断请求。
- 2、是低电平触发, 在低电平时间内中断一直有效; 因此如果在电平没有恢复之前中断程序就已经执行完成从而退出, 那么会在退出后又再次进入中断; 但只要中断没有退出是不会重复触发的。所以, 电平触发方式适合于外部中断以低电平输入而且中断服务能清除外部中断源的情况

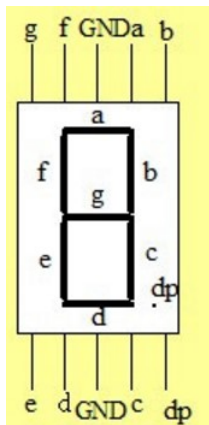
26. 一个共阴极的 8 段 LED 数码管, 要使其显示数字 6 的字形, 则七字形码为()

- A: FDH B: 02H C: 7DH D: 82H

答案: C

分析: 考察数码管的显示方式

知识点:



数码管有 a, b, c, d, e, f, g, dp 八个显示位, 按照二进制的编写方式, 共阴极数码管所有显示位全部点亮可表示为 1111 1111 (FFH), 共阳极数码管全部点亮为 0000 0000 (00H), 而将数字 6 点亮的二进制编写码为 1011 1110, 按照电路连接方式需要自右向左读写二进制, 因此数字 6 的共阴极二进制编码为 0111 1101 (7DH)

27. C 语言程序从()开始执行

- A: 程序中第一条可执行语句
- B: 程序中第一个函数
- C: 程序中的 main 函数
- D: 包含文件中的第一个函数

答案: C

分析: C 语言的基础知识, main 函数是 C 语言程序的入口

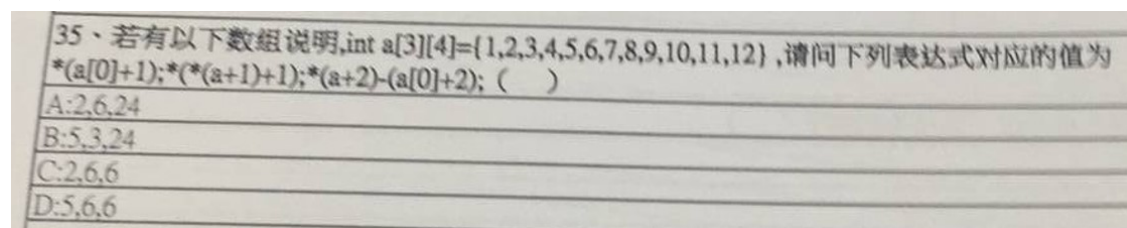
28. 在 C 语言程序中, 表达式 5%2 的结果是()

- A: 2.5
- B: 2
- C: 1
- D: 3

答案: D

分析: 考察运算符%

29.



答案: C

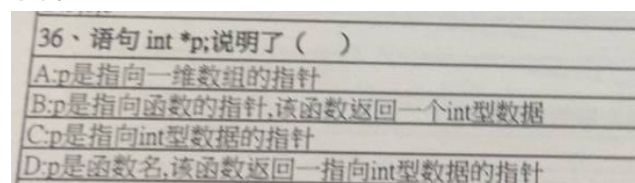
分析: a 是二维数组名, 默认为数组的第一行第一个元素。因此

`*(a[0]+1)`: a[0] 数组第一行, +1 数组第一行第二个元素, *取值, 结果为 2

`*(*(a+1)+1)`: `*(a+1)` 默认是第二行, +1 是第二行第二个元素, 结果为 6

`*(a+2)-(a[0]+2)`: `*(a+2)` 是默认是第三行第一个元素 9, `a[0]+2` 是第一行第三个元素 3, 结果为 6

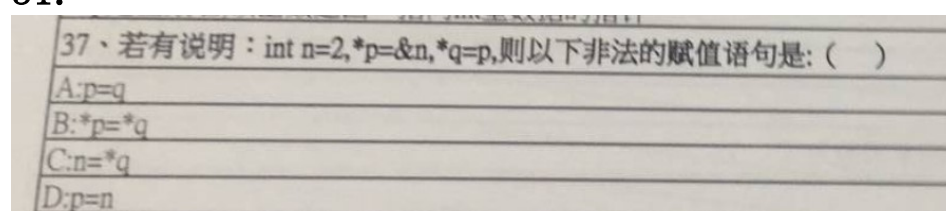
30.



答案: C

分析: 首先 p 是个指针, 在没有对它进行赋值初始化的情况下, 它只是个指针, 指向随机区域, 所以直接排除 A、B、C

31.

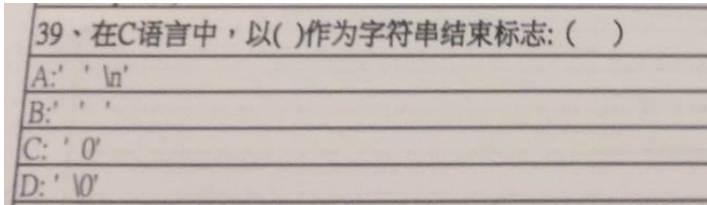


答案: D

分析: 考察变量类型, 赋值的前提是类型匹配, D 选项 p 是指针 n 是整形变量, 类型不匹配无法赋值, 而 *q 或者 *p 运算结果都是整型值所以 B、C 没问题, p、q 都是

指针，所以 A 没问题

32.



答案: D

分析: 考察 C 语言基础-转义字符, A 是换行, B 是空格, C 是字符 0

33. 若进栈的序列为 1、2、3、4，进栈过程中可以出栈，则下列不可能出现的序列是()

- A. 1、4、3、2 B. 2、3、4、1
- C. 3、1、4、2 D. 3、4、2、1

答案: C

分析: 考察栈的先进后出的知识, 例如: ↓入栈 ↑出栈

A. 1 ↓ 1 ↑ 2 ↓ 3 ↓ 4 ↓ 4 ↑ 3 ↑ 2 ↑ → 1、4、3、2

B. 1 ↓ 2 ↓ 2 ↑ 3 ↓ 3 ↑ 4 ↓ 4 ↑ 1 ↑ → 2、3、4、1

D. 1 ↓ 2 ↓ 3 ↓ 3 ↑ 4 ↓ 4 ↑ 2 ↑ 1 ↑ → 3、4、3、1

C. 1 ↓ 2 ↓ 3 ↓ 3 ↑ 2 ↑ 1 ↑ 4 ↓ 4 ↑ → 3、2、1、4 不是 3、1、4、2

34. 下列的变量 Data.B 数值范围是()

struct{ unsigned A:1; unsigned B:2; } Data

- A. 0 或 1 B. 0~255 C. 0~65535 D. 0~3

答案: D

分析: 考察位段概念

unsigned A:1; //1 位, 有效值, 0, 1

unsigned B:2; //2 位, 有效值, 0, 1, 2, 3

35. 请问下列程序运行后 C 变量为()

```
unsigned char A=200; unsigned char B=100; unsigned char C;  
C = A+B;
```

- A. 300 B. 255 C. 45 D. 44

答案: D

分析: 类型的取值范围, unsigned char 型取值范围是 0 ~ 128, 因此 A 的值是 72, A+B 的值是 172-128 = 44

二、简答题

1. 在 win32 下 char, int, float, double 各占多少位?

答案: char: 1, int: 4, float: 4, double: 8

分析: win32 是指 Windows 32 位系统, 因此要回答 32 位系统的类型占位

2. strcpy() 和 memcpy() 的区别?

答案:

1、复制的内容不同。strcpy 只能复制字符串, 而 memcpy 可以复制任意内容, 例如字符数组、整型、结构体、类等。

2、复制的方法不同。strcpy 不需要指定长度, 它遇到被复制字符串的串结束符“\0”才结束, 所以容易溢出。memcpy 则是根据其第 3 个参数决定复制的长度。

3、用途不同。通常在复制字符串时用 strcpy, 而需要复制其他类型数据时则一般用 memcpy

3. 说出字符常量和字符串常量的区别, 并使用运算符 sizeof 计算有什么不同?

答案:

形式上: 字符常量是单引号引起的一个字符;

字符串常量是双引号引起的若干个字符;

含义上: 字符常量相当于一个整型值, 可以参加表达式的运算;

字符串常量代表一个地址值 (该字符串在内存中存放的位置);

占内存大小: 字符常量只占一个字节;

字符串常量占若干个字节, (至少一个字符结束标志) 使用运算符 sizeof 计算多占一字节的存储空间

4. 什么是指针? 谈谈你对指针的理解?

答案: 指针是类型化的地址,

在程序编程中, 一般通过变量的名字访问变量, 这种访问成为直接访问, 实际上, C 语言在编译之后都会把变量名编译成一个十六进制的地址, 这个地址是存储变量的首地址编号, 通过这种方式访问称为间接访问, 指针就是通过间接方式访问变量, 指针是类型化的地址, 指针的基类型跟普通数据类型一样, 但是指针的名称是基类型名称加上*

5. 指针的集中典型应用情况?

答案:

1. 数组名作为函数形参时, 在函数体内, 其失去了本身的内涵, 仅仅只是一个指针

2. 指针数组

3. 函数指针 (数组)

4. 链表

5. 函数传参, 指向某个值或者类

6. 利用指针变量访问数组元素

7. 操作文件

6. class 和 struct 的区别？

答案：c++中 struct 和 class 的主要区别在于默认的存取权限不同，struct 默认为 public，而 class 默认为 private

7. UDP 和 TCP 的区别是什么？

答案：

TCP 服务提供了数据流传输、可靠性、有效流控制、全双工操作和多路复用技术等。

UDP 并不提供对 IP 协议的可靠机制、流控制以及错误恢复功能等。由于 UDP 比较简单，UDP 头包含很少的字节，比 TCP 负载消耗少。

TCP：提供稳定的传输服务，有流量控制，缺点是包头大，冗余性不好

UDP：不提供稳定的服务，包头小，开销小

8. 什么是多态？多态有什么作用？

答案：多态（Polymorphism）按字面的意思就是“多种状态”。概念：同一操作作用于不同的对象，可以有不同的解释，产生不同的执行结果，这就是多态性。简单的说：就是用基类的引用指向子类的对象。

作用主要是两个：

1. 隐藏实现细节，使得代码能够模块化；扩展代码模块，实现代码重用；
2. 接口重用：为了类在继承和派生的时候，保证使用家族中任一类的实例的某一属性时的正确调用。
3. 解决项目中紧耦合的问题，提高程序的可扩展性。

9. 重载和覆盖有什么区别？

答案：

a. 成员函数被重载的特征：

- 1) 相同的范围（在同一个类中）；
- 2) 函数名字相同；
- 3) 参数不同；
- 4) virtual 关键字可有可无。

b. 覆盖是指派生类函数覆盖基类函数，特征是：

- 1) 不同的范围（分别位于派生类与基类）；
- 2) 函数名字相同；
- 3) 参数相同；
- 4) 基类函数必须有 virtual 关键字。

10. 什么是共有继承、受保护继承、私有继承？

答案：

1. 公有继承(public)

公有继承的特点是基类的公有成员和保护成员作为派生类的成员时，它们都保持原有的状态，而基类的私有成员仍然是私有的，不能被这个派生类的子类所访问。

2. 私有继承(private)

私有继承的特点是基类的公有成员和保护成员都作为派生类的私有成员，并且不能被这个派生类的子类所访问。

3. 保护继承(protected)

保护继承的特点是基类的所有公有成员和保护成员都成为派生类的保护成员，并且只能被它的派生类成员函数或友元访问，基类的私有成员仍然是私有的。

11. 说一下 TCP 3 次握手、4 次挥手的全过程。

答案：

第一次握手：Client 将标志位 SYN 置为 1，随机产生一个值 seq=J，并将该数据包发送给 Server，Client 进入 SYN_SENT 状态，等待 Server 确认。

第二次握手：Server 收到数据包后由标志位 SYN=1 知道 Client 请求建立连接，Server 将标志位 SYN 和 ACK 都置为 1，ack (number)=J+1，随机产生一个值 seq=K，并将该数据包发送给 Client 以确认连接请求，Server 进入 SYN_RCVD 状态。

第三次握手：Client 收到确认后，检查 ack 是否为 J+1，ACK 是否为 1，如果正确则将标志位 ACK 置为 1，ack=K+1，并将该数据包发送给 Server，Server 检查 ack 是否为 K+1，ACK 是否为 1，如果正确则连接建立成功，Client 和 Server 进入 ESTABLISHED 状态，完成三次握手，随后 Client 与 Server 之间可以开始传输数据了。

第一次挥手：Client 发送一个 FIN，用来关闭 Client 到 Server 的数据传送，Client 进入 FIN_WAIT_1 状态。

第二次挥手：Server 收到 FIN 后，发送一个 ACK 给 Client，确认序号为收到序号+1（与 SYN 相同，一个 FIN 占用一个序号），Server 进入 CLOSE_WAIT 状态。

第三次挥手：Server 发送一个 FIN，用来关闭 Server 到 Client 的数据传送，Server 进入 LAST_ACK 状态。

第四次挥手：Client 收到 FIN 后，Client 进入 TIME_WAIT 状态，接着发送一个 ACK 给 Server，确认序号为收到序号+1，Server 进入 CLOSED 状态，完成四次挥手。

12. 简述 Cocos2d-x 内存管理机制。

答案：Cocos2d 内存管理的基本原理是对象内存引用计数。当声明定义一个对象时，会在堆上为这个对象分配内存，并且有一个变量 m_uReference 专门用于记录该对象被引用了多少次。当我们在堆上分配一块内存空间的时候，这个对象的引用计数就是 1，当有对象要引用这块内存空间的时候，这个引用计数就增加 1，当有对象不再引用这块内存的时候引用计数就减 1，当这个引用计数减为 0 的时候就使用 delete 删除掉这块内存，这样就做到了当有对象引用的时候会正常的访问这块内存，引用完毕也可以正常的回收。

13. 简述 cocos2d-x 中 CCScene、CCLayer、CCSprite、CCNode

答案：

1. CCScene 中存放的是需要渲染的层、人物角色和菜单等。它们可以作为一个整体，一起渲染，一起销毁，一起被场景切换是用
2. CCLayer 同样是 CCNode 的子类，通常用 addChild 方法把 CCLayer 的子类添加到 CCScene 中。CCScene 提供了一个表演的舞台，而 CCLayer 提供了舞台中的幕布。CCLayer 对象定义了可描绘的区域，定义了描绘的规则，用于定义外观和事件 behavior。
3. CCSprite 对象就是一个可以移动、选择、伸缩、动画的 2d 图形，CCSprite 对象的子成员中可以包含其它的 CCSprite 对象。当 CCSprite 对象转换坐标系的时候，它所包含的 CCSprite 对象也会转换坐标系。CCSprite 是 CCNode 的子类，所以他们也可以使用 actions 来转换坐标
4. CCNode 是 cocos2d-x 中一个很重要的类，CCNode 是场景、层、菜单、精灵等的父类。所有可以被渲染或者包含可渲染的都是一个 CCNode。最主要的 CCNode 有：CCScene, CCLayer, CCSprite, CCMenu。

14. 什么是热更新？Lua 热更新原理是什么？

答案：最简单的解释就是不关软件直接更新，更新期间软件直接用，现在大部分软件是冷更新，要退出才能更新通俗的讲，热更新就是应用里面某些模块和页面是用 h5 js 来写的，就和网页一样，代码都在服务器上，更新只需要更改服务器上的代码就可以了

Lua 的 require 把一个 lua 文件加载存放到 package.loaded[modelname]。当我们加载一个模块的时候，会先判断是否在 package.loaded 中已存在，若存在则返回改模块，不存在才会加载(loadfile)，防止重复加载。package.loaded 是一个 Table，其中包含了全局表_G、默认加载的模块和用户加载的模块。